

Rosmaster R2 Smart Warehouse Project

- Developer Documentation

This document serves as the primary onboarding and operational guide for the Rosmaster R2 (Jetson Nano) stack. It covers Docker management, networking, hardware bringup, ROS 2 interactions, custom node development, and the historical patches applied to the system to achieve its current state.

1. Networking & SSH Access

To control the robot without a monitor, you must connect via Secure Shell (SSH).

Finding the Robot's IP

- **LCD Screen:** The most reliable way is to check the small OLED/LCD screen on the robot chassis upon startup; it will display the current IP address.
- **Terminal:** Alternatively, run `hostname -I` on the Jetson terminal.

Note: Typically, the IP will look like 192.168.1.X or 10.0.0.X depending on your local network.

Connecting via SSH

From your main computer (on the same Wi-Fi):

```
ssh jetson@<JETSON_IP_ADDRESS>
```

- **Username:** jetson
- **Password:** yahboom

2. Docker Management

The ROS 2 Foxy environment is containerized. It is crucial to manage these containers properly so you don't accidentally spin up multiple instances fighting for the same USB ports.

Active vs. Stopped Containers

- **Active (Running):** These containers are live. They occupy system memory (RAM) and maintain active connections to hardware like the LiDAR and Camera. You can use `docker exec` on these.
- **Stopped (Exited):** These containers are "off." They preserve your files, but you cannot enter them until you turn them back on.

Viewing Containers

To see which containers are **currently running**:

```
sudo docker ps
```

To see **ALL containers** (including stopped ones):

```
sudo docker ps -a
```

Entering a Container (The 2-Step Process)

If your container is "Exited" (Status: Exited), you must start it before entering. You can use either the **CONTAINER ID** or the **NAME** (e.g., wonderful_bell).

Step 1: Start the container

```
sudo docker start wonderful_bell
```

Step 2: Execute the shell

```
sudo docker exec -it wonderful_bell /bin/bash
```

Tip: When doing robotics development over SSH, use tmux on the host OS to open multiple panes that all docker exec into the same container. This prevents processes from dying if your connection drops.

3. Environment Setup & Workspaces

Yahboom splits the robot's code into two separate workspaces. **You must source both in every new terminal** before running ROS 2 commands.

Workspace 1: The Library Workspace

- **Path:** /root/yahboomcar_ros2_ws/software/library_ws/
- **Purpose:** Contains low-level hardware drivers (like slidar_ros2 and astra_camera).

Workspace 2: The Main Workspace

- **Path:** /root/yahboomcar_ros2_ws/yahboomcar_ws/
- **Purpose:** Contains the robot's kinematics, SLAM, Navigation, and computer vision logic.

The Sourcing Commands:

```
source /root/yahboomcar_ros2_ws/software/library_ws/install/setup.bash
```

```
source /root/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash
```

4. Hardware Bringup (Starting the Robot)

Run each of these in a separate terminal pane/tmux window.

A. The Base Node (Motor Controller & Odometry)

- **Command:** `ros2 launch yahboomcar_bringup yahboomcar_bringup_R2_launch.py`
- **Function:** Establishes the serial bridge to the STM32 for motor control and /odom publishing.

B. The LiDAR (RPLidar A1)

- **Command:** `ros2 run slidar_ros2 slidar_node --ros-args -p serial_port:=/dev/ttyUSB0 -p serial_baudrate:=115200 -p frame_id:=laser`

- **Note:** Requires 115200 baud rate for the A1 model.

C. The Depth Camera (Orbbec Astra)

- **Command:** `ros2 launch astra_camera astra.launch.xml`

5. ROS 2 CLI Tools & Visualization

Data Verification

- **Check topics:** `ros2 topic list`
- **Check frequency:** `ros2 topic hz /camera/depth/image_raw`
- **Echo odometry:** `ros2 topic echo /odom`

Manual Control

- **Drive Forward:**
`ros2 topic pub -1 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"`
- **Teleop Keyboard:**
`ros2 run teleop_twist_keyboard teleop_twist_keyboard`

Web Visualization (ROSboard)

If you cannot run GUI tools like RViz over SSH:

```
cd /root/rosboard
./run
```

Then visit `http://<JETSON_IP_ADDRESS>:8888` in your laptop browser.

6. Custom Development

Create a 3rd workspace for project-specific logic:

1. `mkdir -p ~/warehouse_ws/src`
2. `ros2 pkg create --build-type ament_python smart_warehouse_localization`
3. Build with `colcon build` and source your new workspace's `install/setup.bash`.

7. System History & Patches Applied

1. USB Device Permissions (Host OS)

- Added udev rule at `/etc/udev/rules.d/56-orbbec-usb.rules` for Astra Camera (Vendor ID 2bc5).
- **User Permissions:** Executed `sudo chmod 666 /dev/ttyUSB*` to allow Docker

non-root access to serial ports.

2. Boot Order Fix: Modified `/boot/extlinux/extlinux.conf` to prioritize the SD Card over the SSD.

3. STM32 Firmware Bootloop (Hardware)

- **Problem:** Original firmware crashed looking for MPU9250 IMU.
- **Fix:** Flashed **V3.5.1** firmware via Windows mcuisp tool to support the **ICM-20948** IMU found on V2.0 expansion boards.